

A Report On
Tetris Game

for

Mini Project 2B: FPGA Based Project
[REV- 2019 C Scheme of Third Year, (TE Sem-VI)]

in

Electronics & Telecommunication Engineering

by

06 DAKSH GHARAT
10 APARNA MANDAL
11 SHRUTI MEHER

Under the Guidance

Mr. Pratik Parsewar



UNIVERSITY OF MUMBAI
AY 2025-2026



Vishnu Waman Thakur Charitable Trust's
VIVA Institute of Technology
Approved by AICTE New Delhi, Recognized by DTE, Govt. of Maharashtra
And Affiliated to University of Mumbai
NAAC "B++" Grade

Department of Electronics and Telecommunication Engineering

CERTIFICATE

This is to certify that the project entitled **Tetris Game** is a bonafide work of

06 DAKSH GHARAT

10 APARNA MANDAL

11 SHRUTI MEHER

submitted to the University of Mumbai in partial fulfillment of the requirement for the award of **Mini Project 2B: FPGA Based Project [REV- 2019 ‘C’ Scheme of Third Year, (TE Sem-VI)] in Electronics & Telecommunication Engineering** as laid down by **University of Mumbai** during academic year **2025-26**

(_____)

Examiner/Reviewer-1

(_____)

Examiner/Reviewer-2

Mr. Pratik Parsewar
Guide

Dr. Archana Ingle
Head of Department

Dr. Arun Kumar
Principal

INDEX

Sr. No.	Name of Topic	Page No.
	List of Abbreviation	i
	List of Figures	ii
1	INTRODUCTION	1
2	LITERATURE SURVEY	2
3	DESIGN METHODOLOGY	
	3.1 BLOCK Diagram	4
	3.2 Flowchart	6
	3.3 Working	8
	3.4 Program	9
4	RESULT & DISCUSSION	24
5	CONCLUSION & FUTURE SCOPE	26
	REFERENCES	27
	Appendix: Data sheets of important components	

LIST OF ABBREVIATIONS

S. No.	ABBREVIATION	DESCRIPTION
1.	FPGA	Field Programmable Gate Array
2.	VGA	Video Graphics Array
3.	HDL	Hardware Description Language
4.	FSM	Finite State Machine
5.	RGB	Red Green Blue (Color Model)
6.	HSYNC	Horizontal Synchronization Signal
7.	VSYNC	Vertical Synchronization Signal
8.	PS/2	Personal System/2 Interface (Keyboard Interface)
9.	CLB	Configurable Logic Block
10.	IOB	Input Output Block
11.	SoC	System on Chip
12.	EDA	Electronic Design Automation
13.	RTL	Register Transfer Level
14.	LUT	Look-Up Table
15.	MHz	Megahertz (Frequency Unit)

LIST OF FIGURES

Fig. No.	Figure Title	Page No.
3.1	Block Diagram of Tetris Game	10
3.2	Flowchart of Tetris Game	11
4.1	Simulation Output of FPGA-Based Tetris Game Using VGA Playground ..	24

INTRODUCTION

The rapid advancement of digital technology has led to the widespread use of Field Programmable Gate Arrays (FPGAs) in designing high-speed and efficient systems. FPGAs provide a flexible platform where hardware can be configured and reconfigured according to application requirements, making them ideal for real-time and parallel processing tasks. Unlike conventional software-based implementations, FPGA-based designs operate at the hardware level, offering improved performance and deterministic behavior. This project explores the implementation of a classic game, Tetris, using an FPGA Spartan-3 board, demonstrating the practical application of digital design concepts in an interactive system.

The proposed system utilizes VGA (Video Graphics Array) interfacing to display the game output on a monitor. The design involves generating precise horizontal and vertical synchronization signals along with RGB color outputs to create visual elements on the screen. The game logic, including the movement and positioning of blocks, is implemented using Verilog HDL and follows synchronous design principles driven by a clock signal. User interaction is achieved through input controls such as push buttons, allowing real-time manipulation of the falling blocks. The entire system operates without the use of a microprocessor, showcasing a pure hardware-based approach to game development.

This project emphasizes key concepts such as VGA signal generation and timing, real-time graphics rendering directly through hardware, and the implementation of game logic using finite state machine-based design techniques. It also demonstrates the importance of clock division and synchronization in ensuring stable system operation. Furthermore, the transition from software-oriented thinking to hardware description language-based design is highlighted, providing valuable insight into embedded system development. Overall, this project serves as a practical learning platform for understanding the integration of digital electronics, hardware design, and real-time system implementation.

LITERATURE SURVEY

The development of FPGA-based gaming systems has gained significant attention due to the increasing demand for high-speed, real-time graphical applications. Earlier game implementations relied on software-based systems, which introduced latency and dependency on processors. The emergence of Field Programmable Gate Arrays (FPGAs) has enabled efficient hardware-based implementation of games with parallel processing capabilities, improved speed, and flexibility.

A comprehensive review of FPGA-based game design was presented by V. Radha Krishna and Rakesh Kumar Y, where various architectures and implementation techniques were analyzed, highlighting the advantages of hardware-based gaming systems [1]. The study provided a strong foundation for understanding efficient FPGA design methodologies applicable to Tetris game implementation.

Jia Li, Qingyuan Zhang, and Yiming Huo demonstrated the implementation of a Tetris game using the Spartan 3E FPGA platform, focusing on VGA interfacing, block rotation logic, and real-time display generation [2]. Their work served as a direct reference for designing the graphical output and control logic in the proposed system.

M. Sanchez-Elez and S. Roman explored the concept of learning hardware design through video game implementation on FPGA platforms, emphasizing modular design and finite state machine (FSM) based control mechanisms [3]. This approach is essential for structuring the game logic in a systematic and efficient manner.

The implementation of graphical games using VGA display was further explored by Roland Szabó and Aurel Gontean through the development of a Pong game on FPGA, which demonstrated synchronization signal generation and display timing control [4]. These concepts are directly applicable to the VGA-based output system used in this project.

Sadiye Nergis Tural Polat presented a fully embedded Tetris game in VHDL, focusing on key functionalities such as block movement, collision detection, and line-clearing algorithms [5]. These elements form the core logic of any Tetris-based system and were crucial in designing the proposed model.

Ramandeep Singh and Nikhil Ahuja proposed a VGA driver implementation using Spartan FPGA, providing insights into signal generation and hardware-level display control [6]. This work contributed to understanding the design of a stable and efficient VGA interface.

Cheng Chih Liu discussed the use of FPGA platforms in digital system design education with gaming applications, highlighting hardware-software partitioning and modular design strategies [7]. This approach supports the structured implementation of complex systems like FPGA-based games.

Akarsha Mishra et al. demonstrated the FPGA implementation of a space shooting game, focusing on real-time control and hardware-based rendering techniques [8]. Their work provided insights into efficient game logic execution and graphical rendering on FPGA platforms.

Overall, these studies highlight the effectiveness of FPGA technology in implementing real-time gaming systems with high performance, modularity, and flexibility. The proposed Tetris game system builds upon these concepts to achieve a fully hardware-based gaming solution.

DESIGN METHODOLOGY

3.1 Block Diagram

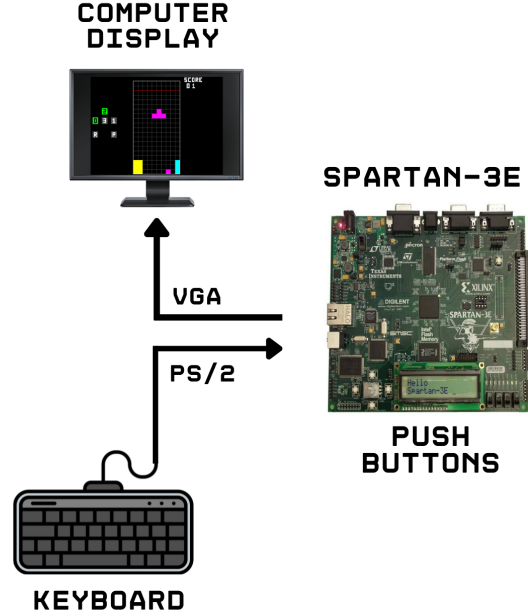


Fig 3.1: Block Diagram of Tetris Game

The design methodology of the proposed system is based on a structured modular architecture, where each block performs a specific function and contributes to the overall operation of the Tetris game. The Spartan-3 FPGA acts as the central processing unit that integrates all modules, including input handling, game logic, and display generation. The system operates on a clock signal, and a clock division mechanism is used to generate the required frequency for VGA timing, ensuring proper synchronization across all modules.

The input to the system is provided exclusively through a PS/2 keyboard, which communicates with the FPGA using serial data transmission. The PS/2 interface module decodes the incoming scan codes and converts them into meaningful control signals such as move left, move right, rotate, and drop. These signals are then passed to the game logic module, where decisions regarding block movement, positioning, and collision detection are made. The game logic maintains the state of the grid, tracks the current falling block, and updates the system based on both user input and automatic game progression.

The rendering module plays a crucial role in converting game data into visual output. It takes inputs such as the current block position, stored grid data, and pixel coordinates from the VGA controller to determine the color of each pixel. The VGA controller generates precise horizontal and vertical synchronization signals along with pixel scanning coordinates, which are used to display the game frame on the monitor. The RGB signals produced by the rendering module are then sent through the VGA interface to the display, resulting in real-time visualization of the game.

All modules are tightly synchronized to ensure smooth and glitch-free operation. The interaction between the PS/2 keyboard input, game logic processing, and VGA output generation forms a continuous loop, enabling real-time gameplay. This modular and hardware-oriented design approach not only improves system efficiency but also demonstrates the effective implementation of digital design concepts such as synchronization, signal processing, and real-time system integration on an FPGA platform.

3.2 Flowchart

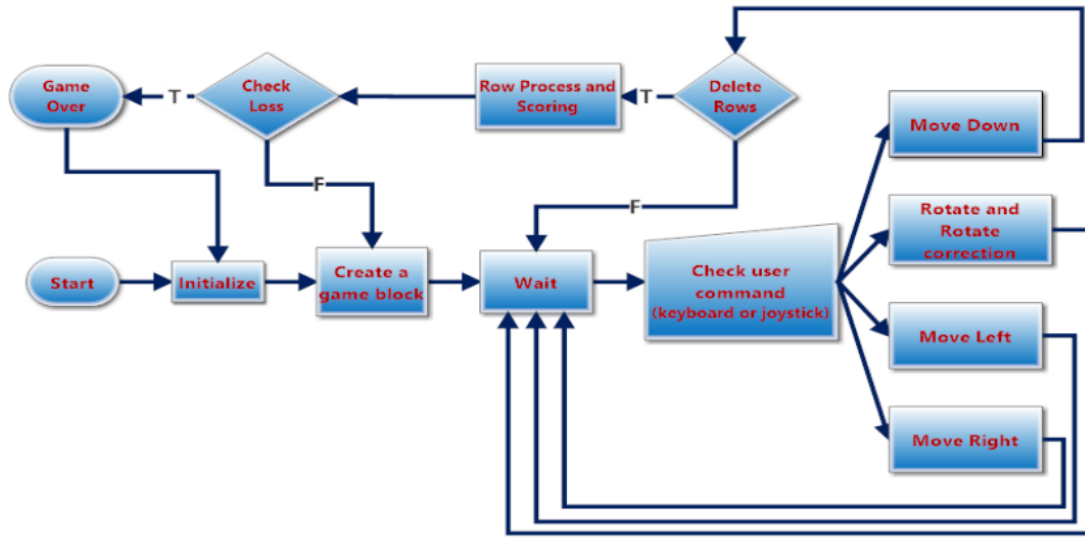


Fig 3.2: Flowchart of Tetris Game

The flowchart illustrates the step-by-step execution of the Tetris game implemented on the FPGA platform. The process begins with the start state, where the system initializes all necessary parameters such as clearing the game grid, resetting the score, and setting the initial position of the falling block. Once initialization is complete, a new game block is created at the top of the grid. The system then enters a waiting state, which acts as a control point for synchronizing game speed and continuously checking for user inputs.

During the waiting phase, the system monitors inputs received from the PS/2 keyboard. Based on the user commands, different operations are performed, such as moving the block left or right, rotating the block, or allowing it to move downward. Each of these actions is processed carefully to ensure that the block does not overlap existing blocks or move outside the defined boundaries of the grid. After executing the input command, the system updates the position of the block and returns to the waiting state, forming a continuous loop that enables real-time interaction.

In parallel with user input handling, the block automatically moves downward after a fixed time interval, simulating gravity. Once the block reaches the bottom of the grid or collides with previously placed blocks, it gets fixed in its position.

The system then checks for any completely filled rows. If such rows are detected, they are deleted, and the blocks above them are shifted downward. At the same time, the score is updated to reflect the cleared rows, enhancing the gameplay experience.

After processing row deletion and scoring, the system evaluates the game status by checking for a loss condition. This occurs when new blocks cannot be placed properly due to lack of space, typically when the stack of blocks reaches the top of the grid. If the loss condition is true, the game transitions to the game over state, stopping further operations. If not, a new block is generated, and the entire process repeats. This looping mechanism ensures continuous gameplay, integrating input handling, movement control, collision detection, scoring, and game termination in an efficient and synchronized manner.

3.3 Working

The working of the proposed system begins with the initialization of the FPGA, where all internal registers, game grid, and control variables are reset to their default states. Once initialized, the VGA controller starts generating synchronization signals and pixel coordinates required to display output on the monitor. Simultaneously, the system activates the PS/2 interface to receive input from the keyboard. A new Tetris block is generated at the top of the grid, and the system enters a continuous execution loop where both display generation and input processing occur in real time.

During operation, the PS/2 keyboard sends scan codes to the FPGA, which are decoded into control signals such as left movement, right movement, and rotation. These inputs are processed by the game logic module, which updates the position and orientation of the falling block while ensuring boundary conditions and collision constraints are satisfied. Along with user input, the block also moves downward automatically at fixed time intervals, simulating gravity. The updated game state is continuously sent to the rendering module, which determines the color of each pixel based on the current grid and block position.

The VGA controller scans through each pixel on the screen and, with the help of the rendering module, displays the corresponding visual output on the monitor. When a block reaches the bottom or collides with previously placed blocks, it becomes fixed in the grid. The system then checks for any completed rows, removes them if present, and updates the game accordingly. This process repeats with the generation of new blocks, creating a continuous gameplay loop. The synchronized operation of input handling, game logic, and VGA output ensures smooth and real-time performance of the Tetris game on the FPGA platform.

3.4 Program

3.4.1 Top Module (project.v)

```
1  'default_nettype none
2
3  module tt_um_vga_example (
4      input  wire [7:0] ui_in,
5      output wire [7:0] uo_out,
6      input  wire [7:0] uio_in,
7      output wire [7:0] uio_out,
8      output wire [7:0] uio_oe,
9      input  wire      ena,
10     input  wire      clk,
11     input  wire      rst_n
12 );
13
14     assign uio_out = 0;
15     assign uio_oe  = 0;
16
17     wire reset = !rst_n;
18
19     wire hsync;
20     wire vsync;
21     wire video_on;
22     wire [9:0] pixel_x;
23     wire [9:0] pixel_y;
24
25     wire [199:0] board_flat;
26     wire [599:0] board_color_flat; // <--- NEW: 600-bit color memory
27     wire [199:0] active_piece_flat;
28     wire [2:0] current_p_id;
29     wire [3:0] score_tens;
30     wire [3:0] score_ones;
31     wire [1:0] R, G, B;
32
33     assign uo_out = {hsync, B[0], G[0], R[0], vsync, B[1], G[1], R[1]};
34
35     vga_sync display_timer (
36         .clk(clk), .reset(reset), .hsync(hsync), .vsync(vsync),
37         .video_on(video_on), .x(pixel_x), .y(pixel_y)
38     );
39
40     tetris_logic game_brain (
```

```

41     .clk(clk), .reset(reset),
42     .btn_left(ui_in[0]), .btn_right(ui_in[1]),
43     .btn_up(ui_in[2]), .btn_down(ui_in[3]),
44     .btn_reset(ui_in[4]), .btn_pause(ui_in[5]),
45     .current_p_id(current_p_id),
46     .active_piece_flat(active_piece_flat),
47     .board_flat(board_flat),
48     .board_color_flat(board_color_flat), // <--- Hooked up
49     .score_tens(score_tens),
50     .score_ones(score_ones)
51 );
52
53 tetris_render pixelPainter (
54     .clk(clk), .video_on(video_on), .x(pixel_x), .y(pixel_y),
55     .current_p_id(current_p_id),
56     .active_piece_flat(active_piece_flat),
57     .board_flat(board_flat),
58     .board_color_flat(board_color_flat), // <--- Hooked up
59     .score_tens(score_tens),
60     .score_ones(score_ones),
61     .btns(ui_in[5:0]),
62     .R(R), .G(G), .B(B)
63 );
64
65 endmodule

```

3.4.2 Game Logic Module (tetris_logic.v)

```
1  'default_nettype none
2
3  module tetris_logic (
4      input wire clk,
5      input wire reset,
6      input wire btn_left,
7      input wire btn_right,
8      input wire btn_down,
9      input wire btn_up,
10     input wire btn_reset,
11     input wire btn_pause,
12     output wire [2:0] current_p_id,
13     output wire [199:0] active_piece_flat,
14     output wire [199:0] board_flat,
15     output wire [599:0] board_color_flat,
16     output reg [3:0] score_tens,
17     output reg [3:0] score_ones
18 );
19
20     reg [22:0] tick_counter;
21     wire [22:0] tick_max = btn_down ? 23'd500_000 : 23'd3_500_000;
22     wire game_tick = (tick_counter >= tick_max);
23
24     reg [3:0] grid_x;
25     reg [4:0] grid_y;
26
27     reg [199:0] board;
28     assign board_flat = board;
29
30     reg [599:0] board_color;
31     assign board_color_flat = board_color;
32
33     reg [2:0] p_id;
34     reg [1:0] rot;
35     assign current_p_id = p_id;
36
37     reg last_left, last_right, last_up, last_down, last_reset,
38     last_pause;
39
40     wire move_left = btn_left ^ last_left;
41     wire move_right = btn_right ^ last_right;
42     wire move_up = btn_up ^ last_up;
```

```

42     wire move_down    = btn_down    ^ last_down;
43     wire do_reset     = btn_reset   ^ last_reset;
44     wire do_pause     = btn_pause   ^ last_pause;
45
46     reg [3:0] dx0, dy0, dx1, dy1, dx2, dy2, dx3, dy3;
47     always @(*) begin
48         dx0=0; dy0=0; dx1=1; dy1=0; dx2=0; dy2=1; dx3=1; dy3=1;
49         case (p_id)
50             0: begin dx0=0; dy0=0; dx1=1; dy1=0; dx2=0; dy2=1; dx3=1;
dy3=1; end
51             1: if (rot[0]==0) begin dx0=0; dy0=0; dx1=1; dy1=0; dx2=2;
dy2=0; dx3=3; dy3=0; end
52                 else begin dx0=0; dy0=0; dx1=0; dy1=1; dx2=0;
dy2=2; dx3=0; dy3=3; end
53             2: case (rot)
54                 0: begin dx0=1; dy0=0; dx1=0; dy1=1; dx2=1; dy2=1; dx3
=2; dy3=1; end
55                 1: begin dx0=1; dy0=0; dx1=1; dy1=1; dx2=2; dy2=1; dx3
=1; dy3=2; end
56                 2: begin dx0=0; dy0=1; dx1=1; dy1=1; dx2=2; dy2=1; dx3
=1; dy3=2; end
57                 3: begin dx0=1; dy0=0; dx1=0; dy1=1; dx2=1; dy2=1; dx3
=1; dy3=2; end
58             endcase
59         endcase
60     end
61
62     wire [4:0] ax0 = grid_x + dx0; wire [4:0] ay0 = grid_y + dy0;
63     wire [4:0] ax1 = grid_x + dx1; wire [4:0] ay1 = grid_y + dy1;
64     wire [4:0] ax2 = grid_x + dx2; wire [4:0] ay2 = grid_y + dy2;
65     wire [4:0] ax3 = grid_x + dx3; wire [4:0] ay3 = grid_y + dy3;
66
67     reg [199:0] active_piece;
68     assign active_piece_flat = active_piece;
69     always @(*) begin
70         active_piece = 200'b0;
71         if (ay0 < 20 && ax0 < 10) active_piece[(ay0*10)+ax0] = 1'b1;
72         if (ay1 < 20 && ax1 < 10) active_piece[(ay1*10)+ax1] = 1'b1;
73         if (ay2 < 20 && ax2 < 10) active_piece[(ay2*10)+ax2] = 1'b1;
74         if (ay3 < 20 && ax3 < 10) active_piece[(ay3*10)+ax3] = 1'b1;
75     end
76
77     wire hit_bottom = (ay0 >= 19) || (ay1 >= 19) || (ay2 >= 19) || (ay3

```

```

77     >= 19);
78     wire hit_block_down = (ay0 < 19 && board[((ay0+1)*10)+ax0]) ||
79                             (ay1 < 19 && board[((ay1+1)*10)+ax1]) ||
80                             (ay2 < 19 && board[((ay2+1)*10)+ax2]) ||
81                             (ay3 < 19 && board[((ay3+1)*10)+ax3]);
82     wire lock_piece = hit_bottom || hit_block_down;
83
84     wire can_move_left = (ax0 > 0 && !board[(ay0*10)+ax0-1]) && (ax1 >
85 0 && !board[(ay1*10)+ax1-1]) &&
86                             (ax2 > 0 && !board[(ay2*10)+ax2-1]) && (ax3 >
87 0 && !board[(ay3*10)+ax3-1]);
88
89     wire can_move_right = (ax0 < 9 && !board[(ay0*10)+ax0+1]) && (ax1 <
90 9 && !board[(ay1*10)+ax1+1]) &&
91                             (ax2 < 9 && !board[(ay2*10)+ax2+1]) && (ax3 <
92 9 && !board[(ay3*10)+ax3+1]);
93
94     integer i, j, dest_row, lines_cleared;
95     reg [199:0] temp_board, next_board;
96     reg [599:0] temp_board_color, next_board_color;
97     reg line_full;
98
99     always @(posedge clk) begin
100         if (reset || btn_reset) begin
101             tick_counter <= 0; grid_x <= 4; grid_y <= 0;
102             board <= 200'b0;
103             board_color <= 600'b0;
104             p_id <= 0; rot <= 0;
105             score_tens <= 0; score_ones <= 0;
106             last_left <= 0; last_right <= 0; last_up <= 0; last_down <=
107 0; last_reset <= 0; last_pause <= 0;
108
109             end else if (!btn_pause) begin
110                 last_left <= btn_left; last_right <= btn_right; last_up <=
111 btn_up;
112                 last_down <= btn_down; last_reset <= btn_reset; last_pause
113 <= btn_pause;
114
115                 if (move_left && can_move_left) grid_x <= grid_x - 1;
116                 if (move_right && can_move_right) grid_x <= grid_x + 1;
117                 if (move_up) begin
118                     rot <= rot + 1;
119                     if (grid_x > 6) grid_x <= 6;
120                 end
121             end
122         end

```

```

113
114     if (game_tick) begin
115         tick_counter <= 0;
116
117         if (lock_piece) begin
118             temp_board = board | active_piece;
119
120             // --- COMPILER-SAFE COLOR SAVING ---
121             temp_board_color = board_color;
122             if (ay0 < 20 && ax0 < 10) begin
123                 temp_board_color[((ay0*10)+ax0)*3 + 0] = (p_id
124 == 0) || (p_id == 2);
125                 temp_board_color[((ay0*10)+ax0)*3 + 1] = (p_id
126 == 1) || (p_id == 2);
127                 temp_board_color[((ay0*10)+ax0)*3 + 2] = 1'b0;
128             end
129             if (ay1 < 20 && ax1 < 10) begin
130                 temp_board_color[((ay1*10)+ax1)*3 + 0] = (p_id
131 == 0) || (p_id == 2);
132                 temp_board_color[((ay1*10)+ax1)*3 + 1] = (p_id
133 == 1) || (p_id == 2);
134                 temp_board_color[((ay1*10)+ax1)*3 + 2] = 1'b0;
135             end
136             if (ay2 < 20 && ax2 < 10) begin
137                 temp_board_color[((ay2*10)+ax2)*3 + 0] = (p_id
138 == 0) || (p_id == 2);
139                 temp_board_color[((ay2*10)+ax2)*3 + 1] = (p_id
140 == 1) || (p_id == 2);
141                 temp_board_color[((ay2*10)+ax2)*3 + 2] = 1'b0;
142             end
143             if (ay3 < 20 && ax3 < 10) begin
144                 temp_board_color[((ay3*10)+ax3)*3 + 0] = (p_id
145 == 0) || (p_id == 2);
146                 temp_board_color[((ay3*10)+ax3)*3 + 1] = (p_id
147 == 1) || (p_id == 2);
148                 temp_board_color[((ay3*10)+ax3)*3 + 2] = 1'b0;
149             end
150
151             next_board = 200'b0;
152             next_board_color = 600'b0;
153             dest_row = 19;
154             lines_cleared = 0;

```

```

148         for (i = 19; i >= 0; i = i - 1) begin
149             line_full = 1'b1;
150             for (j = 0; j < 10; j = j + 1) begin
151                 if (temp_board[(i * 10) + j] == 1'b0)
line_full = 1'b0;
152             end
153             if (!line_full) begin
154                 for (j = 0; j < 10; j = j + 1) begin
155                     next_board[(dest_row * 10) + j] =
temp_board[(i * 10) + j];
156                     next_board_color[((dest_row * 10) + j)
*3] = temp_board_color[((i * 10) + j)*3];
157                     next_board_color[((dest_row * 10) + j)
*3 + 1] = temp_board_color[((i * 10) + j)*3 + 1];
158                     next_board_color[((dest_row * 10) + j)
*3 + 2] = temp_board_color[((i * 10) + j)*3 + 2];
159                 end
160                 dest_row = dest_row - 1;
161             end else begin
162                 lines_cleared = lines_cleared + 1;
163             end
164         end
165
166         if (next_board[4] || next_board[5] || next_board
[14] || next_board[15]) begin
167             board <= 200'b0;
168             board_color <= 600'b0;
169             score_tens <= 0; score_ones <= 0;
170         end else begin
171             board <= next_board;
172             board_color <= next_board_color;
173
174             if (lines_cleared > 0) begin
175                 if (score_ones + lines_cleared >= 10) begin
176                     score_ones <= score_ones +
lines_cleared - 10;
177                     if (score_tens < 9) score_tens <=
score_tens + 1;
178                 end else begin
179                     score_ones <= score_ones +
lines_cleared;
180                 end
181             end
end

```

```
182         end
183
184         grid_y <= 0; grid_x <= 4; rot <= 0;
185         p_id <= (p_id == 2) ? 0 : p_id + 1;
186     end
187     else begin
188         grid_y <= grid_y + 1;
189     end
190 end else begin
191     tick_counter <= tick_counter + 1;
192 end
193 end
194 end
195 endmodule
```

3.4.3 Rendering Module (tetris_render.v)

```
1  'default_nettype none
2
3  module tetris_render (
4      input wire clk,
5      input wire video_on,
6      input wire [9:0] x,
7      input wire [9:0] y,
8      input wire [2:0] current_p_id,
9      input wire [199:0] active_piece_flat,
10     input wire [199:0] board_flat,
11     input wire [599:0] board_color_flat,
12     input wire [3:0] score_tens,
13     input wire [3:0] score_ones,
14     input wire [5:0] btns,
15     output reg [1:0] R,
16     output reg [1:0] G,
17     output reg [1:0] B
18 );
19
20     wire [3:0] draw_grid_x = (x - 220) / 20;
21     wire [4:0] draw_grid_y = (y - 40) / 20;
22     wire is_locked = board_flat[(draw_grid_y * 10) + draw_grid_x];
23     wire is_active = active_piece_flat[(draw_grid_y * 10) + draw_grid_x
24 ];
25
26     // --- COMPILER-SAFE COLOR EXTRACTION ---
27     wire [2:0] locked_color;
28     assign locked_color[0] = board_color_flat[((draw_grid_y * 10) +
29 draw_grid_x) * 3 + 0];
30     assign locked_color[1] = board_color_flat[((draw_grid_y * 10) +
31 draw_grid_x) * 3 + 1];
32     assign locked_color[2] = board_color_flat[((draw_grid_y * 10) +
33 draw_grid_x) * 3 + 2];
34
35     // --- MULTIPLEXED FONT ENGINE ---
36
37     // NEW: "SCORE" text label coordinates!
38     wire is_score_lbl = (x >= 440 && x < 520 && y >= 24 && y < 44);
39     wire [2:0] lbl_idx = (x - 440) >> 4; // Shift by 4 is the same as
40 dividing by 16 for perfect letter spacing!
41
42     wire is_score = (x >= 450 && x < 494 && y >= 50 && y < 70);
```

```

38  wire is_btn_0 = (x >= 50 && x < 62 && y >= 200 && y < 220);
39  wire is_btn_1 = (x >= 130 && x < 142 && y >= 200 && y < 220);
40  wire is_btn_2 = (x >= 90 && x < 102 && y >= 160 && y < 180);
41  wire is_btn_3 = (x >= 90 && x < 102 && y >= 200 && y < 220);
42  wire is_btn_4 = (x >= 50 && x < 62 && y >= 260 && y < 280);
43  wire is_btn_5 = (x >= 130 && x < 142 && y >= 260 && y < 280);
44
45  wire is_any_text = is_score_lbl | is_score | is_btn_0 | is_btn_1 |
is_btn_2 | is_btn_3 | is_btn_4 | is_btn_5;
46
47  wire [3:0] f_x = is_score_lbl ? ((x - 440) - (lbl_idx * 16)) / 4 :
48  is_score ? ((x >= 474) ? (x - 474)/4 : (x - 450)
/4) :
49
50  is_btn_0 ? (x - 50)/4 :
51  is_btn_1 ? (x - 130)/4 :
52  is_btn_2 ? (x - 90)/4 :
53  is_btn_3 ? (x - 90)/4 :
54  is_btn_4 ? (x - 50)/4 :
55  is_btn_5 ? (x - 130)/4 : 0;
56
57  wire [3:0] f_y = is_score_lbl ? (y - 24)/4 :
58  is_score ? (y - 50)/4 :
59  is_btn_2 ? (y - 160)/4 :
60  (is_btn_0 | is_btn_1 | is_btn_3) ? (y - 200)/4 :
61  (is_btn_4 | is_btn_5) ? (y - 260)/4 : 0;
62
63  wire [3:0] f_digit = is_score_lbl ? (
64  lbl_idx == 0 ? 4'd5 : // 'S' (Looks
exactly like 5!)
65  lbl_idx == 1 ? 4'd13 : // 'C'
66  lbl_idx == 2 ? 4'd0 : // '0' (Looks
exactly like 0!)
67  lbl_idx == 3 ? 4'd10 : // 'R'
4'd14 // 'E'
68  ) :
69  is_score ? ((x >= 474) ? score_ones :
score_tens) :
70
71  is_btn_0 ? 4'd0 :
72  is_btn_1 ? 4'd1 :
73  is_btn_2 ? 4'd2 :
74  is_btn_3 ? 4'd3 :
75  is_btn_4 ? 4'd10 :
is_btn_5 ? 4'd11 :

```

```

76         4'd0;
77
78     reg draw_text_pixel;
79     always @(*) begin
80         draw_text_pixel = 0;
81         if (is_any_text && f_x < 3 && f_y < 5) begin
82             case (f_digit)
83                 0: draw_text_pixel = (f_x != 1) || (f_y == 0) || (f_y
84 == 4);
85                 1: draw_text_pixel = (f_x == 1) || (f_y == 4) || (f_x
86 == 0 && f_y == 1);
87                 2: draw_text_pixel = (f_y == 0 || f_y == 2 || f_y == 4)
88 || (f_x == 2 && f_y == 1) || (f_x == 0 && f_y == 3);
89                 3: draw_text_pixel = (f_y == 0 || f_y == 2 || f_y == 4)
90 || (f_x == 2);
91                 4: draw_text_pixel = (f_x == 2) || (f_y == 2) || (f_x
92 == 0 && f_y < 2);
93                 5: draw_text_pixel = (f_y == 0 || f_y == 2 || f_y == 4)
94 || (f_x == 0 && f_y == 1) || (f_x == 2 && f_y == 3);
95                 6: draw_text_pixel = (f_y == 0 || f_y == 2 || f_y == 4)
96 || (f_x == 0) || (f_x == 2 && f_y == 3);
97                 7: draw_text_pixel = (f_y == 0) || (f_x == 2);
98                 8: draw_text_pixel = (f_y == 0 || f_y == 2 || f_y == 4)
99 || (f_x == 0) || (f_x == 2);
100                9: draw_text_pixel = (f_y == 0 || f_y == 2 || f_y == 4)
101 || (f_x == 2) || (f_x == 0 && f_y == 1);
102                10: draw_text_pixel = (f_x == 0) || (f_y == 0) || (f_y
103 == 2 && f_x < 3) || (f_x == 2 && f_y == 1) || (f_x == 1 && f_y == 3)
104 || (f_x == 2 && f_y == 4);
105                11: draw_text_pixel = (f_x == 0) || (f_y == 0) || (f_y
106 == 2) || (f_x == 2 && f_y == 1);
107                13: draw_text_pixel = (f_x == 0) || (f_y == 0) || (f_y
108 == 4); // NEW: Character 'C'
109                14: draw_text_pixel = (f_x == 0) || (f_y == 0) || (f_y
110 == 2 && f_x < 2) || (f_y == 4); // NEW: Character 'E'
111                default: draw_text_pixel = 0;
112            endcase
113        end
114    end
115
116    // --- BUTTON BACKGROUND BOXES ---
117    wire is_box_0 = (x >= 44 && x < 68 && y >= 196 && y < 224);
118    wire is_box_1 = (x >= 124 && x < 148 && y >= 196 && y < 224);

```

```

105     wire is_box_2 = (x >= 84 && x < 108 && y >= 156 && y < 184);
106     wire is_box_3 = (x >= 84 && x < 108 && y >= 196 && y < 224);
107     wire is_box_4 = (x >= 44 && x < 68 && y >= 256 && y < 284);
108     wire is_box_5 = (x >= 124 && x < 148 && y >= 256 && y < 284);
109     wire draw_box_bg = is_box_0 | is_box_1 | is_box_2 | is_box_3 |
is_box_4 | is_box_5;

110
111     wire is_pressed = (is_box_0 && btns[0]) ||
112                       (is_box_1 && btns[1]) ||
113                       (is_box_2 && btns[2]) ||
114                       (is_box_3 && btns[3]) ||
115                       (is_box_4 && btns[4]) ||
116                       (is_box_5 && btns[5]);
117
118     always @(posedge clk) begin
119         if (~video_on) begin
120             R <= 2'b00; G <= 2'b00; B <= 2'b00;
121         end else begin
122
123             if (is_any_text && draw_text_pixel) begin
124                 if (is_pressed) begin
125                     R <= 2'b00; G <= 2'b00; B <= 2'b00;
126                 end else begin
127                     R <= 2'b11; G <= 2'b11; B <= 2'b11;
128                 end
129             end
130
131             else if (draw_box_bg) begin
132                 if (is_pressed) begin
133                     if (is_box_4) begin R <= 2'b11; G <= 2'b00; B <= 2'
b00; end
134                     else if (is_box_5) begin R <= 2'b11; G <= 2'b11; B
<= 2'b00; end
135                     else begin R <= 2'b00; G <= 2'b11; B <= 2'b00; end
136                 end else begin
137                     R <= 2'b01; G <= 2'b01; B <= 2'b01;
138                 end
139             end
140
141             else if (x >= 220 && x < 420 && y >= 40 && y < 440) begin
142                 if (is_active) begin
143                     case (current_p_id)
144                         0: begin R <= 2'b11; G <= 2'b11; B <= 2'b00;

```

```

end
145         1: begin R <= 2'b00; G <= 2'b11; B <= 2'b11;
end
146         2: begin R <= 2'b11; G <= 2'b00; B <= 2'b11;
end
147         default: begin R <= 2'b11; G <= 2'b11; B <= 2'
b11; end
148     endcase
149 end
150 else if (is_locked) begin
151     case (locked_color)
152         3'd1: begin R <= 2'b11; G <= 2'b11; B <= 2'b00;
end
153         3'd2: begin R <= 2'b00; G <= 2'b11; B <= 2'b11;
end
154         3'd3: begin R <= 2'b11; G <= 2'b00; B <= 2'b11;
end
155         default: begin R <= 2'b11; G <= 2'b11; B <= 2'
b11; end
156     endcase
157 end
158 else if (y == 80) begin
159     R <= 2'b11; G <= 2'b00; B <= 2'b00;
160 end
161 else if (
162     x == 220 || x == 240 || x == 260 || x == 280 || x
== 300 ||
163     x == 320 || x == 340 || x == 360 || x == 380 || x
== 400 ||
164     y == 40 || y == 60 || y == 100 || y == 120 || y
== 140 ||
165     y == 160 || y == 180 || y == 200 || y == 220 || y
== 240 ||
166     y == 260 || y == 280 || y == 300 || y == 320 || y
== 340 ||
167     y == 360 || y == 380 || y == 400 || y == 420
168 ) begin
169     R <= 2'b01; G <= 2'b01; B <= 2'b01;
170 end
171 else begin
172     R <= 2'b00; G <= 2'b00; B <= 2'b00;
173 end
174 end

```

```

175
176         else if ( ((x == 219 || x == 420) && (y >= 39 && y <= 440))
177             ||
178                 ((y == 39 || y == 440) && (x >= 219 && x <= 420))
179         ) ) begin
180             R <= 2'b11; G <= 2'b11; B <= 2'b11;
181         end else begin
182             R <= 2'b00; G <= 2'b00; B <= 2'b00;
183         end
184     end
endmodule

```

3.4.4 VGA Sync Module (vga_sync.v)

```
1  'default_nettype none
2
3  module vga_sync (
4      input wire clk,
5      input wire reset,
6      output reg hsync,
7      output reg vsync,
8      output wire video_on,
9      output reg [9:0] x, // Current X pixel
10     output reg [9:0] y // Current Y pixel
11 );
12
13     parameter HD = 640, HF = 16, HB = 48, HR = 96, HMAX = 799;
14     parameter VD = 480, VF = 10, VB = 33, VR = 2, VMAX = 524;
15
16     always @(posedge clk) begin
17         if (reset) begin
18             x <= 0;
19             y <= 0;
20         end else begin
21             if (x == HMAX) begin
22                 x <= 0;
23                 if (y == VMAX) y <= 0;
24                 else y <= y + 1;
25             end else begin
26                 x <= x + 1;
27             end
28         end
29     end
30
31     always @(posedge clk) begin
32         hsync <= ~((x >= (HD + HF)) && (x < (HD + HF + HR)));
33         vsync <= ~((y >= (VD + VF)) && (y < (VD + VF + VR)));
34     end
35
36     assign video_on = (x < HD) && (y < VD);
37
38 endmodule
```

RESULT AND DISCUSSION

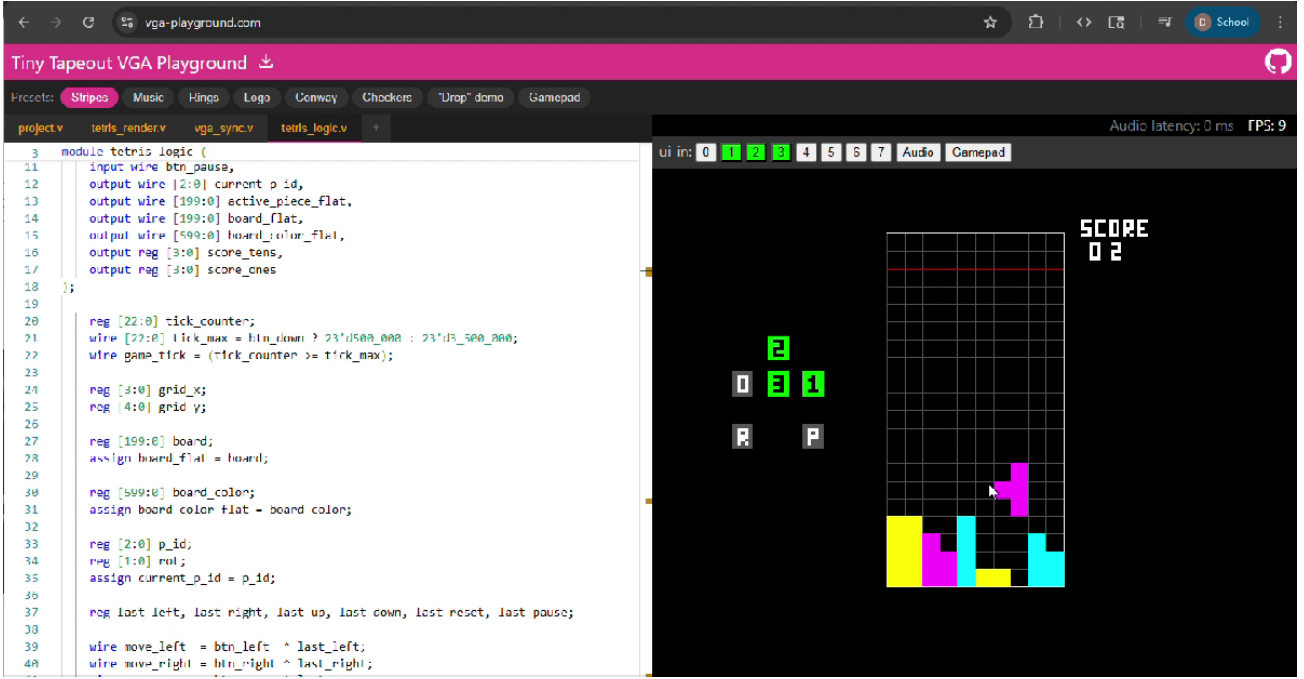


Fig 4.1: Simulation Output of FPGA-Based Tetris Game Using VGA Playground

The simulation of the Tetris game was successfully carried out using the VGA Playground platform, where the design was implemented using four main Verilog modules: the top module (project.v), VGA controller (vga_sync.v), game logic module (tetris_logic.v), and rendering module (tetris_render.v). These modules work together to generate VGA signals, process game logic, and display the output in real time. The VGA Playground environment provides a visual interface where the generated signals are directly mapped to a virtual display, allowing clear observation of the game behavior without requiring physical hardware.

The simulation output shows a properly structured Tetris grid displayed on the screen along with falling blocks of different shapes and colors. The VGA synchronization signals ensure stable display, while the rendering module converts game data into pixel-level RGB outputs. The score is also displayed on the screen, updating dynamically as rows are completed, which confirms correct implementation of scoring logic. The smooth downward movement of blocks indicates proper

timing control using internal counters, and the grid alignment verifies accurate coordinate mapping in the VGA system.

User interaction in the simulation is achieved through keyboard inputs, where specific keys are mapped to control actions. The key '0' is used to move the block to the left, '1' to move it to the right, '2' to rotate the block, and '3' to move it downward faster. Additional controls include 'R' for restarting the game and 'P' for pausing it. The system successfully responds to these inputs in real time, demonstrating correct implementation of input handling and control logic. Overall, the simulation validates the functionality of the complete design, including block movement, rotation, collision handling, scoring, and display generation, confirming that the system is ready for hardware implementation with minor adjustments.

CONCLUSION & FUTURE SCOPE

The implementation of the Tetris game on the Spartan-3 FPGA successfully demonstrates the capability of hardware-based systems in executing real-time graphical applications. The project integrates multiple functional modules such as VGA signal generation, game logic processing, and input handling through a PS/2 keyboard, all working synchronously within the FPGA. The use of Verilog HDL enabled the design of an efficient and modular system, ensuring reliable operation and high-speed performance without relying on any external processor.

The developed system effectively handles block movement, rotation, collision detection, and line-clearing mechanisms, which are essential components of the Tetris game. The VGA interface provides a stable and continuous display output, while the rendering module ensures accurate visualization of game elements. The project also highlights important concepts such as finite state machine design, clock synchronization, and hardware-level parallel processing, making it a valuable learning experience in digital system design.

In the future, the project can be further enhanced by incorporating additional features such as advanced scoring mechanisms, multiple game levels with increasing difficulty, and improved graphical interfaces. Support for different input devices, such as USB keyboards or wireless controllers, can also be explored. Furthermore, the design can be optimized for implementation on modern FPGA platforms with higher resources, enabling more complex game functionalities. The integration of audio output and multiplayer capabilities can also be considered to improve user interaction and overall gaming experience.

REFERENCES

- [1] V. Radha Krishna and Rakesh Kumar Y, "A Review Paper on Games Designed and Implemented on FPGA", International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), 2020.
- [2] Jia Li, Qingyuan Zhang, and Yiming Huo, "Spartan 3E-1200 Based Tetris Game Box", Embedded System Design Advanced Course (EDA385), 2008.
- [3] M. Sanchez-Elez and S. Roman, "Learning Hardware Design by Implementing Video Game on FPGA", Universidad Complutense de Madrid (UCM), Year Not Available.
- [4] Roland Szabó and Aurel Gontean, "Pong Game on an FPGA Development Board Using a Computer Screen as Display", International Journal of Computer Science and Applications (IJCSA), 2015.
- [5] Sadiye Nergis Tural Polat, "A Fully Embedded Tetris Game Application in VHDL", NOHU Journal of Engineering Sciences, 2020.
- [6] Ramandeep Singh and Nikhil Ahuja, "SoC Implementation of VGA Driver Using Spartan 3AN Series FPGA", International Journal of Computer Applications (IJCA), 2013.
- [7] Cheng Chih Liu, "Use of FPGAs in a Digital System Design Course with Computer Gaming Applications", American Society for Engineering Education (ASEE), 2018.
- [8] Akarsha Mishra, Adesh Kumar, and Rakshita Parihar, "Design and FPGA Implementation of Space Shoot Game", International Journal of Control Theory and Applications (IJCTA), Year Not Available.

APPENDIX

1. Spartan-3 FPGA

The Spartan-3 FPGA is a Field Programmable Gate Array developed by Xilinx, widely used for digital system design and prototyping. It consists of configurable logic blocks (CLBs), input/output blocks (IOBs), and embedded memory resources. The device supports high-speed processing and parallel execution of multiple operations, making it suitable for real-time applications such as video signal generation and gaming systems. It operates on a configurable clock and supports hardware description languages like Verilog and VHDL for implementation.

2. VGA Interface

The VGA (Video Graphics Array) interface is used to display graphical output on a monitor. It requires synchronization signals such as horizontal sync (HSYNC) and vertical sync (VSYNC) along with RGB color signals. The standard VGA resolution used in this project is 640×480 at 60 Hz. Proper timing control is essential to ensure stable image display. The FPGA generates these signals to render the Tetris game visually on the screen.

3. PS/2 Keyboard Interface

The PS/2 interface is used for user input through a keyboard. It operates using a serial communication protocol with clock and data lines. The keyboard sends scan codes corresponding to key presses, which are decoded by the FPGA. In this project, the PS/2 keyboard is used to control block movement, rotation, pause, and reset operations in the Tetris game.

4. Clock Signal

The clock signal is a fundamental component of synchronous digital systems. It ensures that all modules operate in coordination. In this project, the FPGA clock is used to control VGA timing, game logic updates, and input processing. Clock division techniques are used to generate slower signals required for game speed control.

5. Display Monitor

A standard VGA-compatible monitor is used to visualize the game output. It receives synchronization and RGB signals from the FPGA and displays the graphical content. The monitor plays a crucial role in verifying the real-time performance of the system.